

1

Basic Notions

1.1 Sets

Basic objects of many branches of mathematics and foundations of computer science are sets and objects that form those sets. The theory of computations on finite state machines (or as we could say the theory of automata) is not an exception. We begin our discussion with an explanation of sets, their objects, constructions that form sets from given ones and notations. We shall use all these notions and notations throughout the book. Here is an informal definition of set.

Definition 1.1.1 *A set is a collection of objects.*

The objects that form the set may be of any type such as positive numbers, rational numbers, some symbols, integers, points in two dimensional space, and even some other sets. The objects that form the given set are called **elements** of the set. We may well call these elements **members** of the set. There are mainly two ways to describe sets. One way of describing a set is simply by listing its elements inside braces. Here are some examples.

Example 1.1.1

1. *The set $\{a, b, c, d\}$ contains four elements: a, b, c, d .*
2. *The set $\{4, 3, 17\}$ consists of three elements, which are 4, 3, 17.*

Given a set and an object we can ask if the object is a member of the set or not. To do this we use two symbols, $\in$ and $\notin$, which tell us if the object is a member

of the set or not, respectively. For example, $a \in \{a, b, c, d\}$, $17 \notin \{a, b, c, d\}$. So the symbols $\in$ and $\notin$ represent the relationship between elements and sets and are notations for set membership and nonmembership, respectively.

Given two sets, say A and B , we can compare them. A natural way to compare two sets is to look at their members and see how members of one set are related to those of the other set. We say that A is a **subset** of B , written $A \subseteq B$, if every member of A is a member of B . In case A is a subset of B but B has some members which are not in A , then we say that A is a **proper subset** of B , written $A \subset B$. Assume that we are given two sets A and B . Then we say that the sets A and B are **equal** if $A \subseteq B$ and $B \subseteq A$. If two sets A and B are equal we write this $A = B$. Thus, in order to show that two given sets A and B are equal, one needs to prove that every member of A is a member of B and every member of B is a member of A .

Exercise 1.1.1 For each of the pair of sets A and B below, determine whether $A \subset B$ or $B \subset A$.

1. $A = \{1, 2\}$, $B = \{2, 3, 4\}$.
2. $A = \{a, c\}$, $B = \{c, d, a\}$.
3. $A = \{3, 2, a\}$, $B = \{2, 3, a\}$.
4. $A = \{4, 7, -1\}$, $B = \{7, 3, 4\}$.

Note that the order of the listing elements of a given set does not matter. Thus the sets $\{a, b, c\}$, $\{b, a, c\}$, $\{c, b, a\}$ are the same. We also note that repetition in the listing of elements of a given set does not matter either. So the sets $\{a, b, b, a, a, c, c, c\}$ and $\{a, c, b\}$ are the same.

An important abstract notion of mathematics is the notion of infinity. A set A is **infinite** if A contains infinitely many members. We cannot, naturally, write down a list of *all* members of an infinite set. Instead we use $\dots$ notation to mean that the list continues forever. A very simple first example of an infinite set is **the set of natural numbers** all of whose elements are listed as follows:

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \dots$$

We use the following notation for this set: ω (read as *omega*). So ω is the set $\{0, 1, 2, 3, \dots\}$. Another example of an infinite set is the set of all **integers**, which we denote by Z :

$$Z = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}.$$

The second way of describing sets consists of specifying its elements. In other words, if we want to describe a set according to some rule or specification, we write

$$\{a \mid a \text{ has property } P\}.$$

So we see that the set $\{n \mid n = 2k \text{ for some natural number } k\}$ consists of all even natural numbers. Similarly, the set $\{n \mid n \text{ is a prime natural number}\}$ consists of all prime numbers.

Now we explain some operations on sets. Operations on sets are means to create new sets from known ones. The most common operations on sets are the **union operation**, **intersection operation**, and **complementation operation**. We now explain these operations.

Let A be a set. Then the **complement** of set A , usually written as $\bar{A}$, is the set whose elements are those under consideration which are not members of A . Say, for example, the objects under consideration are natural numbers. Therefore, if A is the set of all even numbers, then the complement of A is the set of all odd natural numbers. It is often very useful in mathematics to visually represent the concepts defined. For the complementation operation we use a type of picture called a **Venn diagram**. For example, in Figure 1.1 we represent the set A as the unshaded area. The comp

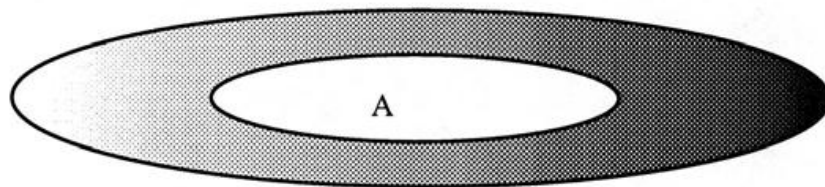


Figure 1.1: The complement of A is the shaded area.

We now describe the **intersection** operation for two sets. Assume that A and B are two sets. Then the intersection of these two sets written $A \cap B$ is obtained by putting all elements that are members of both A and B into one single set. For example, for sets $A = \{3, 4, 17, b, c\}$ and $B = \{a, 3, b, c, d\}$, the intersection of these two sets $A \cap B$ is $\{b, c, 3\}$. Thus we see that $A \cap B$ consists of all elements that are members of both A and B . A **Venn diagram** for the intersection operation is represented in Figure

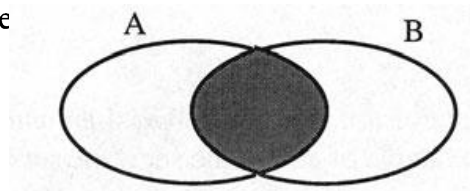


Figure 1.2: The intersection of A and B is the shaded area.

Assume we have two sets: A and B . Let us now take all elements of A and all elements of B , and then put all of these elements into one single set. This is called the **union** of A and B . The union of A and B is written as $A \cup B$. For example, if $A = \{3, 4, 17\}$ and $B = \{a, b, c, d\}$, then the union $A \cup B$ is $\{a, b, c, d, 3, 4, 17\}$.

Thus we see that $A \cup B$ consists of all elements that are either members of A or members of B . Hence, we can write $A \cup B$ as follows:

$$A \cup B = \{x \mid x \text{ is a member of } A \text{ or } x \text{ is a member of } B\}.$$

A **Venn diagram** for the union operation of two sets is represented in Figure 1.3. The union set in Figure 1.3 is the whole shaded area.

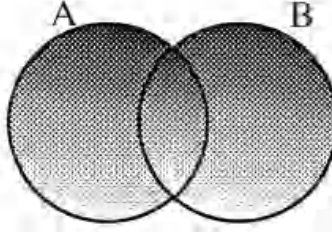


Figure 1.3: The union of A and B is the shaded area.

Note that the set $A \cap B$ is a subset of the set $A \cup B$. This can be seen by examining the Venn diagrams.

Exercise 1.1.2

1. The **difference** of A and B , written $A \setminus B$, is the set of all elements that belong to A but not B . Draw a Venn diagram for the difference of sets A and B .
2. The **symmetric difference** of two sets A and B is the set defined by

$$(A \setminus B) \cup (B \setminus A).$$

Draw a Venn diagram for the symmetric difference of A and B .

Now it is clear that

$$A \cup B = B \cup A, \quad A \cap B = B \cap A, \quad \bar{\bar{A}} = A.$$

The last equality states that the complement of the complement of any set A is the set A itself. One last note: the set with no elements is called **the empty set**. We denote the empty set by $\emptyset$. Clearly, for any set A , we must have

$$A \cup \emptyset = A \quad \text{and} \quad A \cap \emptyset = \emptyset.$$

Exercise 1.1.3 Check the correctness of the equalities

$$A \cup B = B \cup A, \quad A \cap B = B \cap A, \quad \bar{\bar{A}} = A, \quad A \cup \emptyset = A, \quad \text{and} \quad A \cap \emptyset = \emptyset.$$

Exercise 1.1.4 For each of the sets A and B below find $A \cup B$ and $A \cap B$.

1. $A = \{1, 2\}$, $B = \{2, 3, 4\}$.
2. $A = \{a, c\}$, $B = \{c, d, a\}$.
3. $A = \{3, 2, a\}$, $B = \{2, 3, a\}$.
4. $A = \{4, 7, -1\}$, $B = \{7, 3, 4\}$.

1.2 Sequences and Tuples

A **sequence** is obtained when we list some objects. So, for example,

$$1, 2, 3, 4 \quad \text{and} \quad a, c, u, w \quad \text{and} \quad 9, 8, 7, 4, 2, 3, 2, 2, 3$$

are sequences. The length of the first sequence is 4, of the second is 4, and of the last is 9. Thus, the **length** of the sequence is the number of elements that occur in the sequence. Note that some elements in the sequence may occur several times. Moreover, the order in the sequences matters which is in contrast with listing elements of sets. Two sequences are **equal** if they have the same length and the same elements occur in the same order. For example, the sequence 1, 2, 5, 4 is not equal to 1, 2, 4, 5 nor to 1, 2, 3, 5, 4, 4.

A sequence is **infinite** if its length is not finite. For example,

$$2, 1, 4, 3, 6, 5, 8, 7, 10, 11, \dots$$

is an infinite sequence. In order to distinguish finite sequences from infinite sequences, finite sequences are called **tuples**. Tuples are usually written within parentheses. So, for example,

$$(1, 2, 3, 4) \quad \text{and} \quad (a, c, u, w) \quad \text{and} \quad (9, 8, 7, 4, 2, 3, 2, 2, 3)$$

are tuples. A tuple of length 2 is called a **pair** or **2-tuple**, a tuple of length 3 is called a **triple** or **3-tuple**, and in general, a tuple of length k is called a **k -tuple**. Tuples can be elements of other sets. For example, the set of all pairs formed from two symbols a and b is $\{(a, a), (a, b), (b, a), (b, b)\}$.

Assume that A and B are given sets. In mathematics it happens quite often that we want to form the set whose elements are pairs of the type (a, b) where a is a member of A and b is a member of B . Here are two examples.

Example 1.2.1 Let A be $\{1, 2\}$, and let B be $\{1, 2, 3\}$. We form the set $\{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3)\}$. Geometrically, we can represent this set as the set of points in the xy -coordinate system.

Example 1.2.2 Let A be $\{1, 2\}$, and let B be ω . So we can form the set $\{(1, 0), (2, 0), (1, 1), (2, 1), (1, 2), (2, 2), (1, 3), (2, 3), \dots\}$. Geometrically, in the xy -coordinate system, we can represent this set as the set of points of the lines $x = 1$ and $x = 2$ whose y -coordinates are natural numbers.

These two examples naturally lead us to the following definition.

Definition 1.2.1 The **cross product** (or **Cartesian product**) of sets A and B , written $A \times B$, is the set of all pairs such that the first element of each pair is a member of A , and the second element is a member of B .

We can easily generalize the cross product operation as follows. The **Cartesian product** of sets $A_1, \dots, A_k$, written $A_1 \times \dots \times A_k$, is the set of all k -tuples $(a_1, \dots, a_k)$ such that each a_i is a member of A_i , $i = 1, \dots, k$. If all A_i are equal to the same set A , then we write A^k .

Example 1.2.3

1. If $A = \{0, 1\}$, $B = \{a, b\}$, $C = \{(0, 0)\}$, then

$$A \times B \times C = \{(0, a, (0, 0)), (0, b, (0, 0)), (1, a, (0, 0)), (1, b, (0, 0))\}.$$

2. The set $Z^2 = \{(i, j) \mid i, j \in Z\}$ consists of all pairs (i, j) of integers. This is the set of all points in the plane whose x and y coordinates are integers.
3. The set $Z^3 = \{(i, j, k) \mid i, j, k \in Z\}$ consists of all triples (i, j, k) of integers. This is the set of all points in the three dimensional space whose x , y and z coordinates are integers. Geometrically, this is a three dimensional grid.

Exercise 1.2.1 Find the cross products $A \times B$, $A \times A$, and $A \times B \times A$ for the following sets:

1. $A = \{1, 2\}$, $B = \{2, 3, 4\}$.
2. $A = \{a, c\}$, $B = \{c, d, a\}$.
3. $A = \omega$, $B = \{1, 2\}$.
4. $A = \{c, d\}$, $B = \emptyset$.

1.3 Functions, Relations, Operations

As this book studies finite state machines (finite automata), the definition of each of these machines involves in one or in another way the notion of function. So this notion is central to the subject of this book. We also add that the notion of function is indeed central to mathematics. Functions are studied from algebraic, topological, algorithmic, and combinatorial points of view.

Informally, a function is given by some rule. The function takes an input from a set A and according to the rule may or may not produce an output from a set B . No function produces two different outputs for the same input. So if a is an input and b is the corresponding output of the function f , then we write

$$f(a) = b.$$

In this case we say that f **maps** a to b . Functions are sometimes called **mappings**. We can collect all the outputs of the function f and form a single set. This set is called the **range** of the function f . For instance, consider the function which takes a natural number n as an input and outputs the number $2n$. Then the range of this function is the set of all even numbers. We denote the range of f by $\text{Range}(f)$.

Exercise 1.3.1 Consider the functions defined below. Find the range of each of the functions:

1. For any integer n , $f(n) = -n$.
2. For any natural number n , $f(n) = 0$.
3. For any integer n , $f(n) = -1$ if n is odd, and $f(n) = 1$ if n is even.

We denote functions by letters $f, g, h, \dots$. Let f be a function. Let A be the set from which f takes inputs. Let B be the set to which f gives outputs. We write this as

$$f : A \rightarrow B.$$

From the set A now we can collect all elements d such that when f is supplied with d , f gives an output. Let us put all such elements into one set. This set is called the **domain of function** f and is denoted by $\text{Domain}(f)$. Thus, for a function $f : A \rightarrow B$ we have

$$\text{Domain}(f) = \{x \mid x \in A \text{ and } f(x) \in B\}.$$

Similarly,

$$\text{Range}(f) = \{y \mid y \in B \text{ and } f(x) = y \text{ for some } x \in A\}.$$

Example 1.3.1 Consider the function $f : \mathbb{Z} \rightarrow \mathbb{Z}$ defined as follows. For every integer n , if $n > 0$, then $f(n) = 1$; if $n < 0$, then $f(n) = -1$. Then the domain of f consists of all nonzero integers.

Exercise 1.3.2 Consider the functions defined below. Find the domain and the range of each of the functions.

1. The function $f : \omega \rightarrow \omega$ is such that for every natural n , f outputs k only when $n = 2k$ where k is a natural number.

2. The function $f : \omega \rightarrow \mathbb{Z}$ is such that for every integer n , if $n \leq 0$, then $f(n) = -n$; if $n > 5$, then $f(n) = n$.

We can describe functions by using tables. Tables usually list all inputs and outputs of functions. We give some examples.

Example 1.3.2 Consider the function $f : \{a, b, c, d, e\} \rightarrow \{2, 3, 4, 5, 6\}$ whose table is given below.

x	a	b	c	d	e
$f(x)$	3	3	3	5	

The range of this function is $\{3, 5\}$. The domain of this function is $\{a, b, c, d\}$.

We give another example.

Example 1.3.3 Consider the function f from $\{1, 2, 3, 4\} \times \{a, b, c\}$ to $\{0, 1, 2\}$ whose table is given below.

f	1	2	3	4
a	1	1	2	2
b	2	1	1	2
c	1	2	2	1

Clearly, the range of the function is $\{1, 2\}$ and the domain of the function is $\{1, 2, 3, 4\} \times \{a, b, c\}$.

In the last two examples the domains and ranges of the functions are finite sets. Therefore, these functions are finite. Formally, a **finite function** is one whose domain and range are finite sets. Clearly, any function with finite domain is a finite function.

Let A and B be sets. Let f be a function whose domain is the set A , that is $\text{Domain}(f) = A$, and whose range is a subset of B . Of course, there is nothing

in the definition of function that guarantees that the range of f coincides with the whole set B . For example f can send all elements of A onto a given fixed element of B . In the case when the range of f coincides with the whole set B we say that f maps A **onto** B . In this case the function f is called **onto function**. For example the mapping $g : \omega \rightarrow \omega$ given by the rule $g(x) = 2x$, for all $x \in \omega$, does not map the set of all natural numbers ω onto ω . However, g maps ω onto the set $\{x \mid x \in \omega \text{ and } x \text{ is even number}\}$. There are other types of mappings which are of particular interest. Among them are **injections** and **bijections**. A mapping f from A into B is an **injection** if $\text{Domain}(f) = A$ and no two elements from A are mapped into the same element of B . For example, the function $g : \omega \rightarrow \omega$ above injects ω into ω .

Exercise 1.3.3 Show that the mapping $f(x) = x^2$, x is an integer, is not an injection from the set of integers $\mathbb{Z}$ into $\mathbb{Z}$.

A mapping f from A to B is a **bijection** if f is onto and is an injection. Sometimes bijections are called one-to-one functions. Thus, for example the mapping g above from ω to ω is not a bijection. Informally, a bijection from A to B sets up a one-to-one correspondence between the elements of the set A and the elements of the set B .

Exercise 1.3.4 Let A and B be two finite sets with the same number of elements. Show that there exists a bijection from A onto B .

We say that two functions are **equal functions** if the functions have the same domain, and for every input from the domain the functions give the same output. Thus, if we want to check whether or not two functions f and g from A into B are equal, we need to show that $f(a) = g(a)$ for all elements $a \in A$ and that $\text{Domain}(f) = \text{Domain}(g)$.

It is interesting to note that there exists a relationship between subsets of a given set A and functions f which map the set A into the set $\{0, 1\}$ with $\text{Domain}(f) = A$. Thus, any function $f : A \rightarrow \{0, 1\}$ whose domain is A can be associated with the set $B = \{x \mid x \in A, f(x) = 1\}$. The function f is called the **characteristic function** of the set B . In turn, any subset B of A defines a function g as follows. For every $x \in A$, if $x \in B$, then $g(x) = 1$; and if $x \notin B$, then $g(x) = 0$. Thus, the function g as defined equals the characteristic function of the set B .

Now we begin explaining relations and predicates. Suppose we have a set A . Let U be a subset of A . Let us take a member, say a , of the set A . For the element a there are two possibilities with respect to the subset U : a is a member of U ; a is not a member of U . So we conclude that for every element a the fact that a is a member of U is either *true* or *false*. This simply means that U defines in a natural way a function f from the set A into the set $\{\text{true}, \text{false}\}$ as follows:

1. If a is in U , then $f(a) = \text{true}$;
2. If a does not belong to U , then $f(a) = \text{false}$.

Such a function is called a **predicate**. In the case of the subset U , the predicate is **unary** because it states whether the fact “ a belongs to U ” is true or false. Predicates can be binary. A **binary predicate** on set A is simply a function f which outputs *true* or *false* for every (input) pair (a, b) from the set A . A **binary relation** on A is a subset R of A^2 . Every binary predicate f defines the binary relation R_f on A as follows:

a pair (a, b) belongs to R_f if and only if $f(a, b) = \text{true}$.

So we see that for every pair (a, b) , it is true that the pair belongs to the relation R_f if and only if the predicate f outputs *true* when the input is (a, b) . Now let us take a subset R of pairs of the set A . In other words, we take R to be a subset of A^2 . The set R defines in a natural way the function f from the set A^2 to the set $\{\text{true}, \text{false}\}$ as follows.

1. If (a, b) is in R , then $f(a, b) = \text{true}$;
2. If (a, b) is not a member of R , then $f(a, b) = \text{false}$.

We see that R defines its own predicate. Thus, we can identify predicates with relations and relations with predicates.

Example 1.3.4 Say $A = \{0, 1, 2\}$. Consider the following relation:

$$R = \{(0, 1), (0, 2), (1, 2)\}.$$

Then the predicate defined by this relation is

1. The predicate outputs *true* on the pairs $(0, 1), (0, 2), (1, 2)$.
2. The predicate outputs *false* on all other pairs.

Example 1.3.5 Suppose that $A = \{0, 1, 2\}$. Consider the predicate whose value is *true* on pairs $(0, 1), (0, 2), (0, 0), (1, 1)$, and $(2, 2)$ and is *false* for all other pairs. Then the relation R defined by this predicate is simply

$$R = \{(0, 1), (0, 2), (0, 0), (1, 1), (2, 2)\}.$$

Therefore, we identify binary relations with binary predicates. We might be interested not only in unary or binary predicates (relations) but also in predicates that output *true* or *false* values on 3-tuples, 4-tuples, or in general, on k -tuples. Therefore, we give the following definition.

Definition 1.3.1 A **k -ary relation** or **k -ary predicate** R , or (k -place predicate) on set A is a subset of A^k . If a k -tuple $(a_1, \dots, a_k)$ belongs to R , then we say that $R(a_1, \dots, a_k)$ is *true*. If the k -tuple $(a_1, \dots, a_k)$ does not belong to R , then we say that $R(a_1, \dots, a_k)$ is *false*.

An example of a 3-place predicate G_+ on ω is the *graph* of the addition function $+$. Thus, for any triple (n, m, s) of natural numbers the predicate G_+ is true on (n, m, s) if and only if $n + m = s$.

Exercise 1.3.5 *Let A be a set with exactly n elements. How many unary relations does A have? How many binary relations does A have? How many k -ary relations does A have?*

Let A be a set. Then a **unary operation** on this set is any function f whose domain is A and the range is a subset of A . Thus, functions $f : A \rightarrow A$ with $\text{Domain}(f) = A$ are unary operations on the set A . For example, the operation that maps every natural number n into $5n$ is a unary operation on ω . The function $\sin(x)$ that maps every real number x into $\sin(x)$ is a unary operation on the set $\mathbf{R}$ of all reals. A **binary operation** on A is any function f whose domain is A^2 and whose range is a subset of A . Thus, functions $f : A^2 \rightarrow A$ with $\text{Domain}(f) = A^2$ are binary operations on the set A . For example, the operation that maps every pair (n, m) of natural numbers into $5n + m$ is a binary operation on ω . The function $y \times \sin(x)$ that maps every pair (x, y) of real numbers into $y \times \sin(x)$ is a binary operation on the set $\mathbf{R}$ of all reals. In general, an **n -ary operation** (or equivalently **n -place operation**) on a set A is a function whose domain is A^n and whose range is a subset of A . Thus, an n -place operation f sends every n -tuple $(a_1, \dots, a_n)$ of elements of the set A into some element $f(a_1, \dots, a_n)$ of A .

Every n -place operation on a set A defines in a natural way an $n + 1$ -ary predicate on A . Indeed, let $f : A^n \rightarrow A$ be an n -ary operation on A . Define the relation G_f , called the **graph of f** as follows: $(a_1, \dots, a_n, b)$ belongs to G_f if and only if $f(a_1, \dots, a_n) = b$. Clearly, G_f is an $n + 1$ -place predicate on A .

1.4 Equivalence Relations

An important class of binary predicates on a set is the class of relations called equivalence relations. Here is a very simple example of an equivalence relation. Consider the set ω of natural numbers. We say that the natural number i is *equivalent* to number j from ω , written $i \sim j$, if $i - j$ is a multiple of number 4. Note that the following properties hold:

1. Each number i is equivalent to itself. This is because $i - i = 0$ is a multiple of 4.
2. If i is equivalent to j then j is equivalent to i . This is of course true because if $i - j$ is a multiple of 4, then so is $j - i$.
3. If i is equivalent to j and j is equivalent to k then i is equivalent to k . This is also true because $i - k = (i - j) + (j - k)$. Note that the sum of any two numbers which are multiples of 4 is also a multiple of 4. Hence $i - k$ is a multiple of 4.

Thus, we see that $\sim$ is a binary relation on ω , e.g., $\sim$ is the following subset of ω^2 :

$$\{(i, j) \mid i - j \text{ is a multiple of } 4\}.$$

This subset $\sim$ possesses the above three properties. Informally, $\sim$ *identifies* numbers i and j if $i - j$ is a multiple of 4. Now let us take number 0. Then the numbers equivalent to 0 are

$$0, 4, 8, 12, 16, \dots$$

Let us take a number not in the list above, for instance 1. The numbers equivalent to 1 are:

$$1, 5, 9, 13, \dots$$

Similarly, the numbers equivalent to 2 are

$$2, 6, 10, 14, \dots$$

Finally, the numbers equivalent to 3 are

$$3, 7, 11, 15, \dots$$

We denote the set of numbers equivalent to 0, 1, 2, and 3, by $[0]$, $[1]$, $[2]$, $[3]$, respectively. We call these sets **equivalence classes** of the binary relation $\sim$. Now these classes satisfy the following two properties.

1. Every natural number is either in $[0]$ or $[1]$ or $[2]$ or $[3]$.
2. Any two distinct equivalence classes have no elements in common.

In other words, $\sim$ defines a **partition** of the natural numbers. We see that the partition consists of subsets of ω such that the union of these subsets gives the set of all natural numbers. Moreover, these subsets have no elements in common, e.g., they are disjoint.

Based on the example provided above we give the following important definition.

Definition 1.4.1 An **equivalence relation** on a set A is a binary relation R on A that satisfies the following three conditions:

1. R is **reflexive** e.g., for all $x \in A$, $(x, x) \in R$.
2. R is **symmetric** e.g., for all $x, y \in A$, if $(x, y) \in R$, then $(y, x) \in R$.
3. R is **transitive** e.g., for all $x, y, z \in A$, if $(x, y) \in R$ and $(y, z) \in R$, then $(x, z) \in R$.

If $(x, y) \in R$, then we say that x and y are **R -equivalent** or simply **equivalent**.

Thus, the relation $\sim$ considered above is, as we have already shown, an equivalence relation on ω .

Exercise 1.4.1 *Let $n \geq 1$ be a natural number. Say that $i, j \in \omega$ are equivalent, written $i \equiv_n j$, if $i - j$ is a multiple of n . Show that the relation $\equiv_n$ is an equivalence relation on ω .*

Now assume that R is an equivalence relation on set A . Our goal is to partition the set A with respect to R as we partitioned ω into $[0]$, $[1]$, $[2]$, and $[3]$ with respect to $\sim$ relation. Take an element $a \in A$. Consider the set of all elements b of A such that a and b are R -equivalent. We denote this set by $[a]$. Thus, we have

$$[a] = \{b \mid (a, b) \in R\}.$$

Call the set $[a]$ an **equivalence class**. Let us now see what properties these equivalence classes possess.

First of all note that each element a of A is in some equivalence class. Indeed, since R is reflexive, we see that $(a, a) \in R$. Therefore, $a \in [a]$. This simply means that the union of equivalence classes gives the set A . Note that this is similar to the case that ω is a union of the equivalence classes $[0]$, $[1]$, $[2]$ and $[3]$ in the example discussed above.

Second, assume that we take two equivalence classes, say $[a]$ and $[b]$. Our claim is that either $[a]$ coincides with set $[b]$, or $[a]$ and $[b]$ have no elements in common. How can we prove this claim? If $[a]$ and $[b]$ have indeed no elements in common, then the claim is proved trivially. So, the only other thing that could happen is that $[a]$ and $[b]$ have an element in common. Assume that $[a]$ and $[b]$ have an element in common. We want to show that $[a] = [b]$. In order to show that two sets $[a]$ and $[b]$ are the same we need to show that $[b] \subseteq [a]$ and $[a] \subseteq [b]$. Say that c is an element that belongs to both $[a]$ and $[b]$. Then $(a, c) \in R$ and $(c, b) \in R$ by the definition of $[a]$ and $[b]$. Now note that R is transitive. Therefore, $(a, b) \in R$. Therefore, $b \in [a]$, that is $(b, a) \in R$. Now take any $x \in [b]$. Then $(x, b) \in R$. Since R is transitive, we get $(x, a) \in R$. We obtain that $[b] \subseteq [a]$. Similarly, $[a] \subseteq [b]$. So we proved the claim. Thus we have proved the following facts:

1. Each element a of A belongs to some equivalence class. Hence the union of all the equivalence classes is A .
2. Any two equivalence classes either coincide or have no elements in common.

These two statements are in exact correspondence with the statements (1) and (2) about the equivalence relation $\sim$ discussed above.

We will need one extra notion that will be used quite often in the book. Consider all the equivalence classes of an equivalence relation R on a set A . Assume that the number of all the equivalence classes is finite and equals n . Then we say that R has a finite index and the index is n . If the number of equivalence

classes altogether is infinite, then we say that the index of R is infinite. More formally, the **index** of the equivalence relation R is the number of its equivalence classes. We conclude that the relation $\sim$ discussed at the beginning of this section has index 4.

Exercise 1.4.2 Show that the index of the relation $\equiv_n$ defined in Exercise 1.4.1 equals n . What are the equivalence classes of the relation $\equiv_n$?

Exercise 1.4.3 Consider the set ω . Let $0 = a_0 < a_1 < a_2 < \dots$ be an infinite sequence of natural numbers. We say that a pair (x, y) of natural numbers belongs to R if and only if $a_i \leq x < a_{i+1}$ and $a_i \leq y < a_{i+1}$ for some i . Show that R is an equivalence relation. What are the equivalence classes of this relation?

Exercise 1.4.4 Let f be a function from A into B with $\text{Domain}(f)$. Define the relation

$$R_f = \{(x, y) \mid f(x) = f(y)\}.$$

Show that the relation is an equivalence relation on A . What are the equivalence classes of R ? The set R_f is called the **kernel** of the function f .

Exercise 1.4.5 Let A be a set and R be an equivalence relation on A . Show that there exists a function $f : A \rightarrow B$ such that R coincides with the kernel (see Exercise 1.4.4) of f . (Hint: the range of the desired function is the set of equivalence classes of R).

Exercise 1.4.6 Assume that R, S, T stand for “reflexive,” “symmetric,” and “transitive,” respectively. For all distinct $a, b, c \in \{R, S, T\}$ find a binary relation that has property a and b but not c .

1.5 Linearly Ordered Sets

Another example of a binary relation is one that *orders* elements of a given set. Such relations are called **linear orderings**. In order to explain linear orderings on sets, consider the set A that consists of three elements a, b , and c . Suppose that we are asked to put these elements in some *order*. Well, we could order these three elements in six different ways. We present three of them. The first one is

$$a \leq_1 b \leq_1 c;$$

the second one is

$$b \leq_2 c \leq_2 a;$$

the third one is

$$c \leq_3 a \leq_3 b.$$

In the first ordering, which we denoted by $\leq_1$, the order $\leq_1$ puts a as the smallest element, puts c as the biggest element, and b is put between a and c . In the second

ordering, which we denoted by $\leq_2$, the order $\leq_2$ puts b as the smallest element, puts a as the biggest element, and c is put between b and a . Finally, in the third ordering, which we denoted by $\leq_3$, the order $\leq_3$ puts c as the smallest element, puts b as the biggest element and a is put between c and b .

Exercise 1.5.1 Write down the other three orderings of elements a , b , and c .

Let us analyse one of these orders, say $\leq_2$. From the point of view of $\leq_2$, $\leq_2$ claims that it holds the pairs

$$(b, b), (b, c), (b, a), (c, c), (c, a), (a, a).$$

This is because $\leq_2$ orders the elements as follows:

$$b \leq_2 b, b \leq_2 c, b \leq_2 a, c \leq_2 c, c \leq_2 a, a \leq_2 a.$$

We conclude that the ordering $\leq_2$ can be identified with a binary relation on the set $\{a, b, c\}$ that consists of the pairs

$$(b, b), (b, c), (b, a), (c, c), (c, a), (a, a).$$

So we can write the equality

$$\leq_2 = \{(b, b), (b, c), (b, a), (c, c), (c, a), (a, a)\}.$$

Exercise 1.5.2 Identify each of the other (five) orderings of elements a , b , and c as binary relations on the set $\{a, b, c\}$.

Now we make the following notation. For a binary relation R and pair (a, b) we may write aRb to mean $(a, b) \in R$. The considerations above lead us to give the following definition.

Definition 1.5.1 Let A be a set. A binary relation $\leq$ on the set A is called **linear ordering** on A if the following properties of $\leq$ hold true:

1. For all $a \in A$, $a \leq a$.
2. For all $a, b \in A$, if $a \leq b$ and $b \leq a$, then $a = b$.
3. For all $a, b, c \in A$, if $a \leq b$ and $b \leq c$, then $a \leq c$.
4. (**linear order axiom**) For all $a, b \in A$ either $a \leq b$ or $b \leq a$.

If $\leq$ is a linear ordering on the set A , then the set A together with the relation $\leq$, that is, the pair $(A, \leq)$ is a **linearly ordered set**.

Now one can give many examples of linearly ordered sets. Here are some of them.

Example 1.5.1

1. *The natural numbers ω together with the natural ordering $\leq$, that is, $(\omega, \leq)$ is a linearly ordered set.*
2. *The pair $(Q, \leq)$, where Q is the set of rational numbers (fractions) and $\leq$ is the natural ordering of rational numbers, is a linearly ordered set.*
3. *The pair $(Z, \leq)$, where Z is the set of integer numbers and $\leq$ is the natural ordering on integer numbers, is a linearly ordered set.*
4. *The pair $(R, \leq)$, where R is the set of all real numbers and $\leq$ is the usual order on R , is a linearly ordered set.*

Of course, not every relation on a set A is a linear ordering of the set. For example, any equivalence relation on A cannot be linear ordering of A unless A contains exactly one element.

Exercise 1.5.3 *Let A be a finite set. Show that there is a binary relation $\leq$ on A such that $(A, \leq)$ is a linearly ordered set.*

Interestingly, one can *add* linearly ordered sets. Indeed, assume that we have two linearly ordered sets $L_1 = (L_1, \leq_1)$ and $L_2 = (L_2, \leq_2)$. Suppose that $L_1 \cap L_2 = \emptyset$. Then the linearly ordered set $L_1 + L_2$, called the sum of L_1 and L_2 , is obtained as follows.

1. The set of all elements of $L_1 + L_2$ is the union $L_1 \cup L_2$.
2. The order $\leq$ on $L_1 + L_2$ is defined as the union

$$\leq_1 \cup \leq_2 \cup \{(a, b) | a \in L_1, b \in L_2\}.$$

Thus, according to the definition above $L_1 + L_2 = (L_1 \cup L_2, \leq)$. An informal explanation of $L_1 + L_2$ is as follows. The relation $\leq$ orders the union of L_1 and L_2 in such a way that elements of L_1 come first in their order and then the elements of L_2 come in their order. Thus, all elements of L_1 are less than all elements of L_2 . It is not hard to see that $L_1 + L_2$ is a linearly ordered set. Indeed, let us show that $L_1 + L_2$ satisfies all the conditions needed for $\leq$ to be a linear ordering (see Definition 1.5.1). Let us check the first condition. Take any $a \in L_1 \cup L_2$. If $a \in L_1$, then $a \leq_1 a$. If $a \in L_2$, then $a \leq_2 a$. Hence, examining how $\leq$ is defined, we see that $a \leq a$.

Let us check the second condition. Suppose that $a \leq b$ and $b \leq a$. Note that in this case if $a \in L_1$, then $b \in L_1$. Similarly, if $a \in L_2$, then $b \in L_2$. Then in the first case $a \leq_1 b$ and $b \leq_1 a$. Then it must be the case that $a = b$. In the second case $a \leq_2 b$ and $b \leq_2 a$. Then it must be the case that $a = b$. So the second condition holds.

We leave the reader to check the correctness of the other axioms of linearly ordered sets for $L_1 + L_2$.

Exercise 1.5.4 Show that $\leq$ defined for $\mathbb{L}_1 + \mathbb{L}_2$ satisfies the following condition. For all $a, b, c \in L_1 \cup L_2$, either $a \leq b$ or $b \leq a$, and if $a \leq b$ and $b \leq c$ then $a \leq c$.

Exercise 1.5.5 Fix a natural number $n > 1$. Consider the $\equiv_n$ equivalence relation (see Exercise 1.4.1). For any number $m \in \omega$, let $r(m)$ be the remainder obtained by dividing m to n . Define the following binary relation $\leq_n$ on the set ω . For any two natural numbers k and s

1. if $k \equiv_n s$ then $k \leq_n s$ if and only if $k \leq s$, where $\leq$ is the natural order.
2. if k is not $\equiv_n$ -equivalent to s , then $k \leq_n s$ if and only if $r(k) \leq r(s)$.

Show that $(\omega, \leq_n)$ is a linearly ordered set.

The next exercise shows that one can *multiply* linearly ordered sets.

Exercise 1.5.6 Let $\mathbb{L}_1 = (L_1, \leq_1)$ and $\mathbb{L}_2 = (L_2, \leq_2)$ be linearly ordered sets. Define the linearly ordered set $\mathbb{L}_1 \times \mathbb{L}_2$ as follows:

1. The elements of $\mathbb{L}_1 \times \mathbb{L}_2$ are all pairs (a, b) such that $a \in L_1$ and $b \in L_2$.
2. The relation $\leq$ on $L_1 \times L_2$ is defined by the following rule. Let $(a_1, b_1), (a_2, b_2) \in L_1 \times L_2$. Then $(a_1, b_1) \leq (a_2, b_2)$ if and only if either $a_1 = a_2$ and $b_1 \leq_2 b_2$, or $a_1 \leq_1 a_2$ and $a_1 \neq a_2$.

Show that the pair $\mathbb{L}_1 \times \mathbb{L}_2 = (L_1 \times L_2, \leq)$ is a linearly ordered set. This is called **the product of linearly ordered sets**.

Exercise 1.5.7 A linearly ordered set $(A, \leq)$ is **well-ordered** if for every nonempty subset B of A there exists a $b \in B$ such that $b \leq b'$ for all $b' \in B$. Do the following:

1. Show that any finite linearly ordered set is well-ordered.
2. Show that $(\omega, \leq)$ is well-ordered.
3. Show that if $\mathbb{L}_1$ and $\mathbb{L}_2$ are well-ordered sets, then so are $\mathbb{L}_1 + \mathbb{L}_2$ and $\mathbb{L}_1 \times \mathbb{L}_2$.

1.6 Partially Ordered Sets

Another important class of binary relations is the class of partial orderings. Linear orderings are examples of partial orderings. Informally a partial ordering of a set compares elements of the set. Some elements a and b may be comparable and the others not. Hence the word “partial” is used. Before we give a definition of partially ordered sets we present one example.

Let $A = \{a, b, c\}$. Consider all subsets of A . These are:

$$\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, A.$$

We can *compare* these sets using the subset relation. The empty set $\emptyset$ is included in all subsets of A . Hence, in this sense, it is the smallest set. The set A contains all subsets of A . Hence, in this sense, A is the biggest set. Similarly, $\{a\}$ is less than $\{a, b\}$ and $\{a, c\}$ because $\{a\}$ is included in both. Likewise, $\{b\}$ is less than $\{b, c\}$ and $\{b, a\}$ since $\{b\}$ is included in both. The sets $\{a\}$ and $\{b\}$ are not comparable because neither of them is a subset of the other. By the same reason the sets $\{a, b\}$ and $\{b, c\}$ are not comparable. Let us make all these explanations more formal now. On the set of all subsets of A consider the relation $\subseteq$ which consists of the following pairs:

$$(X, X), (X, A), (\emptyset, X), (\{a\}, \{a, b\}), (\{a\}, \{a, c\}), (\{b\}, \{b, c\}), (\{b\}, \{b, a\}), \\ (\{c\}, \{c, b\}), (\{c\}, \{c, a\}),$$

where $X \subset A$. The relation $\subseteq$ on subsets of A satisfies the following properties:

1. For all X , $X \subseteq X$.
2. For all X and Y , if $X \subseteq Y$ and $Y \subseteq X$, then $X = Y$.
3. For all X, Y, Z , if $X \subseteq Y$ and $Y \subseteq Z$, then $X \subseteq Z$.

Informally, $\emptyset$ is the smallest element, $\{a, b, c\}$ is the greatest element with respect to $\subseteq$ relation. A set X is smaller than Y if $X \subseteq Y$. There are incomparable subsets of A , for example $\{a\}$ and $\{b\}$ are incomparable. Thus, we are led to the following definition.

Definition 1.6.1 Let B be a set. A binary relation $\leq$ on the set B is called a **partial ordering** on B if the following properties of $\leq$ hold:

1. For all $a \in B$, $a \leq a$.
2. For all $a, b \in B$, if $a \leq b$ and $b \leq a$, then $a = b$.
3. For all $a, b, c \in B$, if $a \leq b$ and $b \leq c$, then $a \leq c$.

If $\leq$ is a partial ordering on the set B , then the set B together with the relation $\leq$, that is, the pair $(B, \leq)$ is a **partially ordered set**.

Thus, one can see that this definition lacks the linear order axiom used in the definition of the linearly ordered sets.

The example above leads us to define another important notion, the notion of power set.

Definition 1.6.2 Let A be any set. The **power set** of A , denoted either by $P(A)$ or by 2^A , is the set whose elements are exactly all subsets of A .

Note that for any set A , $\emptyset$ and A are elements of $P(A)$. Therefore, $P(A)$ is not empty.

Exercise 1.6.1 Write down the power set of the set $\{0, 1, 2, 3\}$.

Exercise 1.6.2 Let A be a set. Show that the following are partially ordered sets.

1. The pair (A, R) , where $R = \{(a, a) | a \in A\}$.
2. The pair $(P(A), \subseteq)$, where $P(A)$ is the power set of A and $X \subseteq Y$ if and only if X is a subset of Y .

Exercise 1.6.3 Let $A = \{a, b, c\}$. How many partial orders are on the set A ?

Exercise 1.6.4 On the set ω consider the relation

$$R = \{(0, n) | n \in \omega\} \cup \{(x, y) | y = k \cdot x \text{ for some } k \in \omega\}.$$

Show that R is a partial order on ω .

Exercise 1.6.5 On the set ω consider the function $f(x) = 2x$. Let $f^i(x)$ be the value obtained by applying f consecutively i times to x . Thus, for example, $f^5(1) = f(f(f(f(f(1)))))) = 32$, $f^2(7) = f(f(7)) = 28$. $f^0(x)$ is always x for all x . Consider the relation

$$R = \{(n, m) | n \in \omega, m \in \omega, \text{ and } f^i(n) = m \text{ for some } i\}.$$

Show that the relation R is a partial order.

Finally, we present a few notions useful in the study of partially ordered sets. Let $(A, \leq)$ be a partially ordered set and let $B \subseteq A$. An element $b_0 \in B$ is **minimal** if no element of B is strictly less than b_0 , that is for all $b \in B$ if $b \leq b_0$, then $b = b_0$.

Example 1.6.1 Let X be the set of all nonempty subsets of set $Y \neq \emptyset$. Consider the partially ordered set $(X, \subseteq)$. The minimal elements of the partially ordered set are exactly those subsets of Y which contain one element only.

An element a_0 in a partially ordered set $(A, \leq)$ is the **least** (or equivalently the **smallest**) element if all elements of A are less than or equal to a_0 , that is, $a_0 \leq a$ for all $a \in A$. It is not hard to see that a partially ordered set cannot have two distinct least elements. Indeed, if a and b are least elements, then it must be the case that $a \leq b$ and $b \leq a$. Therefore, a must be equal to b . On the other hand, one can give an example of a partially ordered set which has distinct minimal elements.

1.7 Graphs

The next fundamental notion needed for this book is the notion of a graph. There are two types of graphs: directed graphs and undirected graphs. We begin our explanation with undirected graphs.

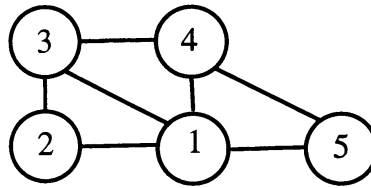


Figure 1.4: An undirected graph example.

An undirected graph contains points and some lines that connect those points. The points are usually called **nodes** or **vertexes**. The first example of a graph is in Figure 1.4.

In the graph presented in Figure 1.4 we have five nodes. We also have seven lines connecting pairs of some nodes. These lines are called **edges**. So this graph has seven edges. There is no sense of direction on the edges. In this graph the nodes are the numbers 1, 2, 3, 4, 5. If we think of the nodes as locations and the edges as paths between locations, then for example we can travel between the nodes 1 and 4 in either direction along the edges connecting them. This graph possesses one topological property: for any two nodes we can travel from one to the other along the edges. For example, we can travel from 1 to 2 in many ways: we can travel from the node 1 to the node 2 using the edge connecting them; we can travel from 1 to 2 by first going to 4, then from 4 to 3 and from 3 to 2. The graphs at which one can travel from any node to any other node along the edges are usually called **connected graphs**.

Let us consider now the second example of a graph presented in Figure 1.5. In this graph we have nine nodes. We also have nine edges between some points. Again there is no sense of direction on the edges. For example, we can travel between points 2 and 3 in either direction. However, this graph is not connected. This is because there is no way we can travel from the node 0 to the node 6.

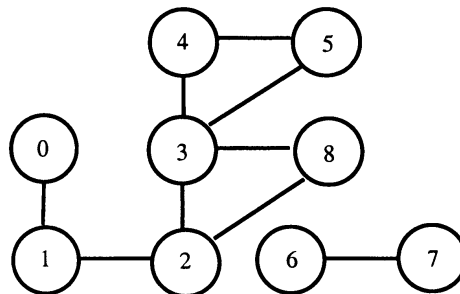


Figure 1.5: Unconnected graph example.

As we see from the examples above, each undirected graph consists of:

1. A set of nodes, and

2. A set of edges.

Now if s and s' are nodes of a graph and they are connected via an edge, then the edge can simply be identified with the pair (s, s') . Since we can travel between s and s' in either direction, we can also identify the edge with the pair (s', s) .

Thus, the first example of an undirected graph can be explained in mathematical terms as follows. The nodes of the graph are:

1, 2, 3, 4, 5,

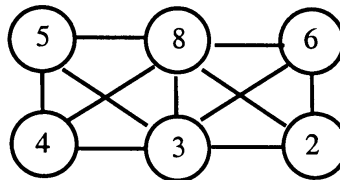
and the edges are

$(1, 2), (2, 1), (1, 3), (3, 1), (1, 4), (4, 1), (1, 5), (5, 1), (2, 3), (3, 2), (3, 4), (4, 3), (4, 5), (5, 4)$.

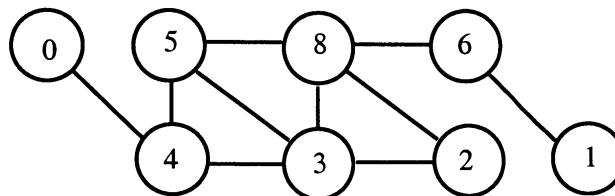
So we see that we can identify every undirected graph with a set S of nodes together with a set E of edges which is a symmetric binary relation on S . So here is a definition.

Definition 1.7.1 An **undirected graph** is a pair (S, E) such that S is a set of nodes and E is a symmetric binary relation on S . If $(s, s') \in E$, then we say that there is an edge between s and s' .

Exercise 1.7.1 Consider the following two undirected graphs:



and



Write down the set of nodes of each of the graphs and identify the edges of the graphs with appropriate symmetric binary relations.

In this book the finite state machines we will study will be presented as **directed graphs**. The difference between undirected and directed graphs is quite obvious. The edges in directed graphs carry information about directions. Here we begin with some examples. Consider the graph in Figure 1.6.

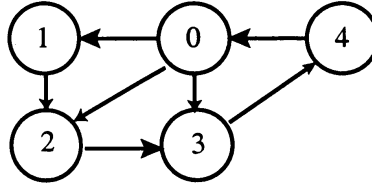


Figure 1.6: A directed graph example.

In this graph we have five nodes. We also have seven lines between some points, again called **edges**. So in this graph we have seven edges. But now each edge possesses a sense of a direction. For example, there are directed edges from point 4 to the node 0, from 0 to the nodes 1, 2 and 3. However, there are directed edges neither from 0 to 4 nor from 1 to 0. Here is the second example presented in Figure 1.7.

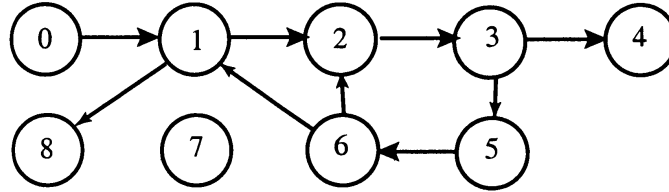


Figure 1.7: A directed graph example.

In this graph we have nine nodes. We also have nine edges between some nodes. Each edge is a directed edge. For example we can travel from the node 2 to the node 3 and from 3 to 5 via edges. We can also travel from the node 0 to 8 by moving along the edges first from 0 to 1 and then from 1 to 8. However, we cannot travel from node 8 to any other node since there is no directed edge outgoing from 8.

The first example (see Figure 1.6) can be explained in mathematical terms as follows. The nodes of the graph are: 0, 1, 2, 3, 4 and the edges are

$$(0, 1), (0, 2), (0, 3), (1, 2), (2, 3), (3, 4), (4, 0).$$

So we see that we can identify every directed graph with a set S of nodes together with a set E which is simply a binary relation on S . So here is a definition.

Definition 1.7.2 A **directed graph** is a pair (S, E) such that S is a set of nodes and E is a binary relation on S . If $(s, s') \in E$, then we say that there is a directed edge from s to s' .

A **path** in a graph (directed or undirected) is a sequence of nodes connected by edges. More formally, a path is a sequence $a_1, \dots, a_k$ of nodes in the graph such that $(a_i, a_{i+1}), i = 1, \dots, k-1$, are all edges of the graph. Note that a path in a directed graph should follow the directions on edges. A graph is **connected** if there exists

a path between any two nodes of the graph. Note that the first example (above) of a directed graph is connected while the second is not connected. Sometimes connected directed graphs are called **strongly connected**.

When we have a graph, we sometimes label edges. Such graphs are called labeled graphs. The labeling of edges usually carries an important piece of information. For example nodes of a graph can represent cities and edges can represent bus routes between the cities. Let us consider an example of a labeled graph presented in Figure 1.8.

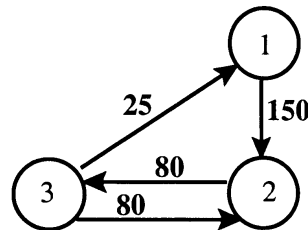


Figure 1.8: A labeled graph example.

The graph in Figure 1.8 is a labeled directed graph. For instance, the edge from 1 to 2 is labeled with **150**. This simply can be interpreted as that the distance (or cost of the ticket) from city 1 to city 2 equals **150**. Let us consider another example presented in Figure 1.9.

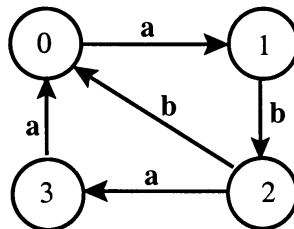


Figure 1.9: Another labeled graph example.

In this example, the graph contains four nodes 0, 1, 2 and 3, and edges (0, 1), (1, 2), (2, 3), (2, 0), (3, 0). The edges are labeled with letters **a** or **b**. One can think of this graph as follows. The nodes represent states of a computer. Labels represent instructions (or inputs). A directed edge from s to s' labeled with, say **a**, represents the following statement. If the computer is in state s and executes the instruction **a** (or reads the input **a**), then the computer changes its state from s to s' .

It is also interesting to note that the labeled graphs define 3-ary relations in a natural way. Let us consider the first example of the directed graph above. Consider the nodes 1 and 2. The edge between these nodes is labeled with **150**. We can simply code the information that there is an edge from 1 to 2 labeled with **150** as the triple:

$$(1, 150, 2).$$

So the whole description of the first directed labeled graph can be presented as:

1. The set $\{1, 2, 3\}$ of nodes, and
2. The set T of triples:

$$\{(1, \mathbf{150}, 2), (2, \mathbf{80}, 3), (3, \mathbf{80}, 2), (3, \mathbf{25}, 1)\}.$$

Similarly, the second directed labeled graph can be presented as:

1. The set S of nodes $\{0, 1, 2, 3\}$, and
2. The set T of triples

$$\{(0, a, 1), (1, b, 2), (2, a, 3), (2, a, 0), (3, a, 0)\}.$$

1.8 Induction

Induction plays an important role in computer science as well as in mathematics. It is quite often that we build certain mathematical or abstract computer science objects by using induction. By constructing the objects, we often want these objects to satisfy certain properties. Then in order to prove that the built objects satisfy the desired properties, we use induction. We recall the induction principle for natural numbers.

Let P be a property on natural numbers. Assume that 0 satisfies the property P . Suppose that for every natural number n , the fact that n satisfies the property P always implies that $n + 1$ satisfies the property P . Then all natural numbers have the property P .

This principle shows us how to check whether a given property P is true for all natural numbers. To check this we must do the following.

- We need to show that 0 satisfies the property P . This is called **the base case** of the induction.
- Assuming that n has property P we should show that $n + 1$ has the property P . The assumption that n has the property P is called the **induction hypothesis**. The process of checking whether or not $n + 1$ has the property P is called **the inductive step**.

Let us consider an application of this principle. Let P be the property that states the following:

For every natural number n , the double sum of all natural numbers less than or equal to n is $n \times (n + 1)$.

In order to check whether this property holds true on all natural numbers, we use the induction principle.

Base Case. Clearly, 0 satisfies the property P because the double sum of all natural numbers less than or equal to 0 is $0 + 0$, and equal to $0 \times (0 + 1)$.

Inductive Step. We assume that the inductive hypothesis holds true. In other words, we suppose that P is true for the natural number n , that is, the double sum of all natural numbers less than or equal to n is $n \times (n + 1)$. Take the number $n + 1$. Now we want to show that the double sum of all natural numbers less than or equal to $n + 1$ is $(n + 1) \times (n + 2)$. Well, the double sum of all natural numbers less than or equal to $n + 1$ equals the double sum of all natural numbers less than or equal to n plus $n + 1$ plus $n + 1$. Since we assume, by the induction hypothesis, that the double sum of all natural numbers less than or equal to n is $n \times (n + 1)$, we see that the double sum of all natural numbers less than or equal to $n + 1$ is

$$n \times (n + 1) + (n + 1) + (n + 1)$$

which in turn equals

$$(n + 2) \times (n + 1).$$

So the property P is true for the number $n + 1$. By the induction principle the property P holds true for all numbers n .

Exercise 1.8.1 *Show, using the induction principle on natural numbers, that the sum $2^0 + 2^1 + 2^2 + \dots + 2^n$ equals $2^{n+1} - 1$ for all n .*

The induction principle on natural numbers can be extended to other objects of mathematics. Such more general induction is used in mathematics and computer science very often. A typical and more general case of the use of induction can informally be described as follows.

We build certain structures, let us call them *objects*, by stages. At the initial stage (we call it stage 0), we pick up certain simple structures and declare them to be objects. At any other stage we build new structures “by putting together” those objects that we have built so far up to the current stage. Then we declare these new structures to be objects too. Then we look at all the objects constructed at stage 0, stage 1, stage 2, $\dots$ and put all of them into one set. Anything from the set is of course an object that is created at certain stage s .

Now if we want to prove some properties of these built objects, then we use induction. The induction is used in a way very similar to the induction principle for natural numbers. Here is an informal explanation of the induction principle.

Let P be a property of the objects we have constructed. Assume that all objects built at the initial stage satisfy the property P . Suppose that for any s , the fact that all objects built at stage i , $i = 0, 1, \dots, s$, satisfy the property P always implies that the objects built at stage $s + 1$ satisfy the property P . Then all the built objects have the property P .

This principle shows us how to check whether a given property P is true for all the objects. To check this we must do the following.

- We need to show that all objects built at stage 0 satisfy the property P . This is called **the base case** of the induction.
- Assuming that all objects built at stages $i, i = 0, \dots, s$, have property P , we should show that all objects built at stage $s + 1$ also have the property P . The assumption that all objects built at stage $i, i = 1, \dots, s$, have the property P is called the **induction hypothesis**. The process of checking whether or not all objects built at stage $s + 1$ have the property P is called the **inductive step**.

Now we give an example. We build objects called **propositions** by stages. The propositions are built as follows.

Stage 0. At this stage propositions are simply symbols

$$p_0, p_1, p_2, \dots, p_n, \dots$$

Call these propositions **atomic propositions**.

Stage $s + 1$. Suppose that α and β are propositions that have been built at the previous stages. Then

$$(\alpha \bigvee \beta), (\alpha \& \beta), (\alpha \rightarrow \beta), (\neg \alpha)$$

are also propositions.

Thus, for example the following are propositions:

$$p_1, (p_1 \bigvee p_2), (\neg p_3), ((\neg p_3) \bigvee (p_1 \& p_5)).$$

The proposition p_1 is defined at *stage 0*, the propositions $(p_1 \bigvee p_2)$ and $(\neg p_3)$ are defined at *stage 1*, and the proposition $((\neg p_3) \bigvee (p_1 \& p_5))$ is defined at *stage 2*. Now we give an application of the induction principle to show that certain properties of propositions hold true.

Let us for example show that every proposition is either atomic or begins with a left parenthesis (and ends with a right parenthesis). We prove this claim by the induction principle.

Base Case. Consider *stage 0* of the definition of propositions. If α is a proposition defined at this stage, then α is atomic. Hence the claim we wanted to prove is correct at the base case.

Inductive Step. Our induction hypothesis is the following. Any proposition α defined before the *stage $s + 1$* is either atomic or begins with a left parenthesis

(and ends with a right parenthesis). Take a proposition α . If α is defined before stage $s + 1$, then, by the induction hypothesis, α is either atomic or begins a left parenthesis (and ends with a right parenthesis). Suppose that α is defined at stage $s + 1$. Then α is in one of the following forms:

$$(\alpha_1 \vee \beta_1), (\alpha_1 \& \beta_1), (\alpha_1 \rightarrow \beta_1), (\neg \alpha_1)$$

Clearly, each of these propositions begins with a left parenthesis (and ends with a right parenthesis). Hence the induction step is proved.

Thus, by the induction principle, every proposition is either atomic or begins with a left parenthesis (and ends with a right parenthesis).

In this book the induction principle will be used quite often, and we suggest the reader to go through the exercises below.

Exercise 1.8.2 *Prove that the power set $P(A)$ of a set A with n elements has 2^n members.*

Exercise 1.8.3 *Consider the following three symbols: a, b, c . Define by stages strings of these symbols:*

Stage 1. *Strings of length 1 are a, b and c only.*

Stage $s + 1$. *Let $u_1, \dots, u_m$ be strings of length s . Then the strings of length $s + 1$ are $u_1 a, u_1 b, u_1 c, \dots, u_m a, u_m b, u_m c$.*

Show that for every natural number $s \geq 1$ the number of strings of length s is 3^s .

Exercise 1.8.4 *Let α be a proposition and $x \in \{\vee, \&, \rightarrow\}$. Show that if $\alpha = (\alpha_1 x \alpha_2)$ and $\alpha = (\beta_1 x \beta_2)$, then $\alpha_1 = \beta_1$ and $\alpha_2 = \beta_2$. Show that if $\alpha = (\neg \alpha_1)$ and $\alpha = (\neg \beta_1)$, then $\alpha_1 = \beta_1$.*

1.9 Trees and König's Lemma

Let us first describe a tree informally. A tree has its root. The root is the lowest node of the tree. Apart from the root the tree has other nodes. Each node may branch up. Say x is a node on the tree. If x has n outgoing branches, then x has exactly n immediate successors which x branches to. Each of these immediate successors may also have outgoing branches. So a general picture of a tree looks like Figure 1.10.

In this figure r is the root. The nodes of the tree are all labeled with letters $a, b, \dots, o$ together with the root. So, for example a has exactly one immediate successor which is d ; b has two immediate successors which are e and f ; i has no outgoing branch so it has no immediate successors. A **path** in this tree is any sequence

$$y_0, y_1, y_2, \dots$$

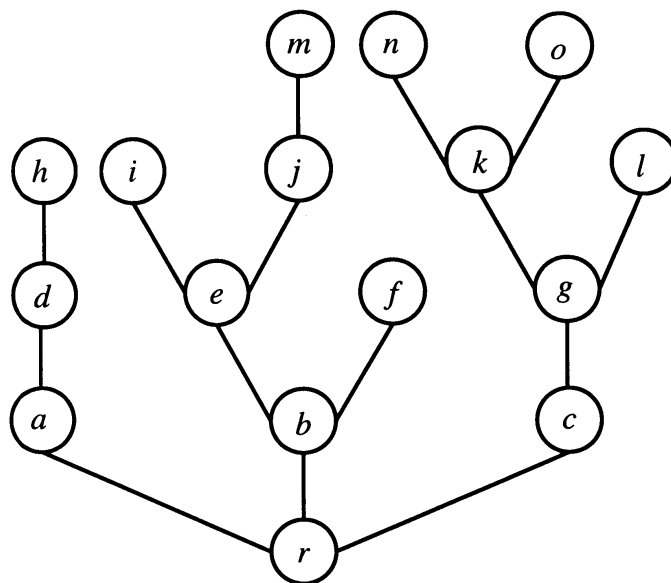


Figure 1.10: An example of a tree.

such that y_0 is the root, y_1 is an immediate successor of y_0 , and in general each y_{i+1} is an immediate successor of y_i . A path may terminate at a node y_j if y_j has no outgoing branches. Thus, for example, in the tree above r, b, e, j, m is a path. So a natural question is the following:

Can we give an exact mathematical definition of a tree?

Well, if we use our knowledge about partially ordered sets, then of course we can give a definition to trees. Indeed, the tree above can be viewed as a partially ordered set where $x \leq y$ if the node x is “below” the node y on the tree. In particular if $x \leq y$, then x and y must belong to the same path. For example, in the picture above $r \leq a, r \leq o, e \leq m, c \leq k, a \leq h$. On the other hand, e and f , d and k , and j and o are incomparable. Now we give a formal definition of a tree.

Definition 1.9.1 A tree is a partially ordered set $T = (T, \leq)$ which has the following properties:

1. It has the root r which is the smallest element in T .
2. For each $x \in T$ the set of all **predecessors of x** $\{y \mid y \leq x\}$ is finite, contains r , and is linearly ordered.

An **immediate successor of x** is a y distinct from x such that $x \leq y$ and there is no element strictly between x and y . Among the trees of particular interest are trees known as finitely branching trees. These trees appear in many areas of mathematics and computer science since, very often, they help us to visualise and reason about structures that occur in mathematical and computer science.

Definition 1.9.2 A tree is **finitely branching** if every node of the tree has finitely many immediate successors.

Above we explained the notion of path informally. Here is a formal definition of a path through a tree.

Definition 1.9.3 A **path** on a tree $(T, \leq)$ is a subset $X \subseteq T$ such that the following two conditions hold:

1. The set X together with the original ordering $\leq$, that is, the pair $(X, \leq)$, is a linearly ordered set.
2. For any subset $Y \subset T$ if $X \subseteq Y$ and $(Y, \leq)$ is a linearly ordered set, then $X = Y$.

Thus, briefly, a path in a tree is a linearly ordered subset of the tree that cannot be properly extended to another linearly ordered subset of T .

Exercise 1.9.1 Show that if a path on a tree is finite, then it contains a node that has no immediate successors.

We are now ready to present a result known as König's Lemma. The proof of the lemma involves reasoning that uses induction, and we ask the reader to study it until it is well understood. It will be used later in the book. The lemma shows us the relationship between an infinite finitely branching tree and infinite paths of the tree. We recall that a tree or a path on a tree is infinite if it has infinitely many nodes.

Lemma 1.9.1 (König's Lemma) A finitely branching tree $T = (T, \leq)$ is infinite if and only if it has an infinite path.

Proof. If $T = (T, \leq)$ contains an infinite path, then clearly T has infinitely many elements. Hence T is infinite.

Suppose now that T is infinite. We want to show that T has an infinite path. We build the desired path by stages. At each stage we put some node into the path we build. The node will be an immediate successor of the node defined at the previous stage.

Stage 0. We put the root r into the path X we want to build. We let $x_0 = r$.

Stage 1. Let $y_1, \dots, y_n$ be all immediate successors of x_0 . There must exist a y_i , the set of all nodes above which is infinite. Otherwise, it is not hard to see that T would be a finite tree. Let x_1 be one of these y_i s that has infinitely many nodes above itself.

Stage $s + 1$. Assume that the sequence $x_0 \leq x_1 \leq \dots \leq x_s$ has been built. Assume that x_1 is an immediate successor of x_0 , and in general x_{i+1} is an immediate successor of x_i , $i \leq s - 1$. Also assume that the set of all nodes above x_s is infinite.

Now we define x_{s+1} as follows. Let $z_1, \dots, z_t$ be all immediate successors of x_s . There must exist a z_i , the set of all nodes above which is infinite. Otherwise, it is not hard to see that the set of nodes above x_s would be finite. This would contradict the induction hypothesis that the set of nodes above x_s is infinite. Set x_{s+1} to be one of these z_i s that has infinitely many nodes above itself.

Thus, we have a sequence

$$x_0, x_1, x_2, \dots$$

of nodes that satisfy the following conditions:

1. For each x_s in the sequence the set of nodes above x_s is an infinite set.
2. The sequence is infinite.
3. Each x_{s+1} is an immediate successor x_s , $s \geq 1$.

We conclude that the sequence $x_0, x_1, x_2, \dots$ is an infinite path on the tree T . We have proved the lemma.

Exercise 1.9.2 Give an example of an infinite tree, all paths of which are finite.

1.10 Countable and Uncountable Sets

In dealing with sets, one naturally encounters the problem of measuring the sizes of sets. For finite sets this problem is solved easily. We just count the number of elements of a given finite set and call the resulting number the size of the set. Thus, two finite sets have the same size if both have the same number of elements. However, it is not so easy to introduce the measure of infinite sets. Indeed, for an infinite set, if we begin counting its elements then we will never finish the process of counting. Therefore, we cannot use the idea of counting to measure the sizes of infinite sets.

In order to measure sizes of infinite sets, let us consider the following two sets. The first set is the set of all even natural numbers:

$$0, 2, 4, 6, 8, \dots$$

The second set is the set of all odd numbers:

$$1, 3, 5, 7, 9, \dots$$

These two sets are infinite and hence both are *larger* than any finite set. However, intuitively, these two sets have the *same size* because the mapping $f(2x) = 2x + 1$, where x is a natural number, is a bijection that *maps* the set of even numbers onto the set of all odd numbers.

This example suggests the idea of measuring the sizes of sets without making any reference to counting the elements of the sets. Thus, for example two finite sets have the same number of elements if and only if the elements of the one set can be paired with the elements of the other set so that the pairing induces a bijective mapping. So formally we now give the following definition.

Definition 1.10.1 *We say that two sets A and B have **the same size** if there exists a bijection f from A onto the set B . If A and B have the same size, then we say that A and B are **equivalent**. In this case we can also say that A **has the size of** B .*

Thus, as we saw above, the set of all even natural numbers has the same size as the set of all odd numbers. Another example is the set of all natural numbers and the set of all even natural numbers. They have the same size because the mapping $f(x) = 2x$ is a bijection from the set of natural numbers onto the set of even numbers. Now we prove some simple facts about the sizes of sets.

Proposition 1.10.1 *Let A, B, C be sets. Then the following are true:*

1. A is equivalent to A .
2. If A and B are equivalent, then B and A are also equivalent.
3. If A and B are equivalent and B and C are equivalent, then A and C are equivalent.

Proof. Clearly, A is equivalent to itself because the *identity mapping*, $id_A(x) = x$, where $x \in A$, is a bijection from A onto A . To prove the second part, assume that A and B are equivalent. Then there exists a bijection f from A onto B . Consider the **inverse of f** , denoted by f^{-1} , and defined by $f^{-1}(b) = a$ if and only if $f(a) = b$. The reader can easily check that f^{-1} is a bijection from B onto A . Hence B and A are equivalent. In order to prove the last part, assume that g and h are bijections from A onto B , and from B onto C , respectively. Then the **composition** of g and h , denoted by gh and defined as $gh(a) = g(h(a))$ for all $a \in A$, is a bijection from A onto C . The proposition is proved.

Exercise 1.10.1 *Show that the following pairs of sets have the same size:*

1. The set of all positive natural numbers and the set of all negative integers.
2. The sets $A = \{x \in \omega \mid x \text{ is a multiple of } n\}$ and $B = \{x \in \omega \mid m \text{ is a multiple of } m\}$, where n and m are fixed positive integers.
3. The set of all natural numbers and the set of all rational numbers (a number is **rational** if it is of the form $p : q$, where $q \neq 0$ and q, p are integers).
4. The sets ω and ω^2 .
5. The sets ω and $\mathbb{Z}^2$.
6. The set ω and the set of all finite subsets of ω .

1.10.1 Countable Sets. A class of sets which is of particular interest is the class of countable sets. Informally, a countable set is one whose elements can be enumerated. For example, we can enumerate the set ω of all natural numbers:

$$0, 1, 2, 3, 4, \dots$$

in their natural order. Note that one can enumerate the elements of ω in a different way, as for example $1, 0, 3, 2, 5, 4, 7, 6, \dots$ is another enumeration. We can also enumerate the set Z of all integers as follows:

$$0, 1, -1, 2, -2, 3, -3, 4, -4, \dots$$

Of course, the elements of every finite set can also be enumerated. Informally, a countable set is one whose elements can be enumerated. Here is a formal definition.

Definition 1.10.2 *A set A is **countable** if A is either finite or has the size of ω .*

As we have already seen the set of even natural numbers is countable. It is also not hard to see that the set of integers is countable. Indeed the mapping $f(2x) = x$ and $f(2x + 1) = -x$ is a bijection from the set ω onto Z . Hence Z is countable.

So, if a set A is countable, then either A is finite or we can write down a sequence

$$a_0, a_1, a_2, a_3, \dots$$

of *all* elements of A so that $a_i \neq a_j$ for all $i \neq j$. Indeed, countability of A implies that there exists a bijection f from ω onto A . Letting a_i be $f(i)$, where $i \in \omega$, we can list *all* elements of A in the sequence

$$a_0, a_1, a_2, a_3, \dots$$

so that $a_i \neq a_j$ for all $i \neq j$. Conversely, if we can list *all* elements of an infinite set A into the sequence

$$a_0, a_1, a_2, a_3, \dots$$

so that $a_i \neq a_j$ for all $i \neq j$, then by setting $f(i) = a_i$, where $i \in \omega$, we establish a bijection f from ω onto A . This shows that A is countable.

We call any sequence

$$a_0, a_1, a_2, a_3, \dots$$

a **sequence without repetition** if $a_i \neq a_j$ for all $i \neq j$. Now we are ready to prove several results about countable sets.

Theorem 1.10.1 *If sets A and B are countable, then so are the sets $A \cup B$ and $A \cap B$.*

Proof. We can assume that both A and B are infinite. Since A and B are countable sets we can list all elements of A and B in the following two sequences without repetitions:

$$a_0, a_1, a_2, a_3, \dots$$

and

$$b_0, b_1, b_2, b_3, \dots$$

We want to list all elements of $A \cup B$ in a sequence

$$c_0, c_1, c_2, c_3, \dots$$

without repetitions. One idea is to consider the sequence

$$a_0, b_0, a_1, b_1, a_2, b_2, \dots$$

The problem with this sequence is that if $A \cap B \neq \emptyset$, then some a_i is equal to a b_j . Therefore, this sequence may not be a sequence without repetitions. To overcome this difficulty we “filter” the sequence

$$a_0, b_0, a_1, b_1, \dots$$

using the following process. The process proceeds by stages. At stage s , we define c_s and ensure that the sequence

$$c_0, c_1, c_2, \dots$$

is without repetition so that every element of $A \cap B$ occurs exactly once. Informally, at even stages of the process we deal with elements of A and at odd stages we deal with elements of B . If the element we deal with at a given stage has not appeared in the list $c_0, c_1, \dots$ that has been constructed so far, then the element is put into the list. Otherwise, we go on to the next stage. Here is a formal description.

Stage 0. Let $c_0 = a_0$.

Stage $s = 2k + 1$. Consider b_k . If b_k has already appeared in the sequence

$$c_0, \dots, c_{s-1}$$

then go on to the next stage. Otherwise, let $c_s = b_k$.

Stage $s = 2k + 2$. Consider a_{k+1} . If a_{k+1} has already appeared in the sequence

$$c_0, \dots, c_{s-1}$$

then go on to the next stage. Otherwise, let $c_s = a_{k+1}$.

It is not hard to see that all elements of $A \cup B$ appear in the sequence

$$c_0, c_1, c_2, c_3, \dots$$

This sequence by construction contains no repetitions of elements.

Now we prove the second part of the theorem. We need to show that $A \cap B$ is countable. If $A \cap B$ is finite, then $A \cap B$ is countable by definition. Assume now that $A \cap B$ is infinite. We will build the sequence

$$c_0, c_1, \dots$$

without repetition that will contain all elements from $A \cap B$. We do it by stages. At even stages we deal with elements of A and at odd stages we deal with elements of B .

Stage 0. If $a_0 \in B$, then set $c_0 = a_0$. Otherwise, go to the next stage.

Stage $s = 2k + 1$. Suppose that we have built a finite sequence

$$c_0, c_1, \dots, c_{t_s-1}.$$

(Note that the sequence can be empty). Consider the element b_k . If $b_k \in A$ and b_k has not appeared in the list we have built so far, then put b_k into the list by setting $c_{t_s} = b_k$. Otherwise, go on to the next stage.

Stage $s = 2k + 2$. Suppose that we have built a finite sequence

$$c_0, c_1, \dots, c_{t_s-1}.$$

Consider the element a_{k+1} . If $a_{k+1} \in B$ and a_{k+1} has not appeared in the list we have built so far, then put a_{k+1} into the list by setting $c_{t_s} = a_{k+1}$. Otherwise, go on to the next stage.

It is not hard to see that all elements of $A \cap B$ appear in the sequence

$$c_0, c_1, c_2, c_3, \dots$$

This sequence by construction contains no repetitions of elements. The theorem is proved.

Theorem 1.10.2 *If A is a countable set, then any subset B of A is countable.*

Proof. If B is finite, then B is certainly countable. Assume that B is infinite. Since A is countable and infinite we can list all elements of A without repetition. Let

$$a_0, a_1, a_2, \dots$$

be such a list. Consider the subsequence

$$b_0, b_1, \dots$$

of all elements of the sequence $a_0, a_1, a_2, a_3, \dots$ that are in B . Clearly, this subsequence is a sequence without repetition so that all elements of B appear in the subsequence. Hence B is a countable set. This proves the theorem.

Exercise 1.10.2 Prove by using Theorem 1.10.2 that if A and B are countable sets, then $A \cap B$ is also countable.

Exercise 1.10.3 Let $A_0, A_1, A_2, \dots$ be a countable sequence of countable sets. Show that the union of all these sets

$$A_0 \cup A_1 \cup A_2 \cup \dots$$

is a countable set.

Exercise 1.10.4 Show that if A and B are countable sets, then $A \times B$ is also a countable set.

Exercise 1.10.5 Let A be a countable set. Show that the set of all finite subsets of A is a countable set.

Exercise 1.10.6 Show that the set of all rational numbers is a countable set.

1.10.2 Diagonalization and Uncountable Sets. In this subsection we show that infinite and uncountable sets exist. This was proved by Cantor. In fact Cantor proved that the set of all real numbers is not countable. The method of the proof is known as the *Diagonalization Method*. The method and its generalizations are used in mathematics and computer science very often. For example, in computability theory the ideas of the diagonalization method are used to construct sets with different properties.

In order to explain the diagonalization method, we begin with a definition of a real number. A **real number** is one that has a decimal representation. Thus, all the rational numbers and the numbers π, e are examples of real numbers. Now we prove that the set of all real numbers is not countable.

Theorem 1.10.3 The set $\mathbf{R}$ of all real numbers is uncountable.

Proof. The proof is by contradiction. We assume that the set $\mathbf{R}$ of all real numbers is countable and show that this assumption leads us to a contradiction.

So, assume that the set $\mathbf{R}$ of all real numbers is countable. Hence we can list *all* real numbers as a sequence

$$r_0, r_1, r_2, \dots$$

without repetition. Now in order to arrive at a contradiction we construct a real number x that will be distinct from any number in the sequence. Of course, this will constitute a contradiction since x must occur in the sequence. We explain this in more detail.

In order to construct a number x distinct from all real numbers, we need to ensure that x satisfies the following requirement

$$R_i: \quad x \neq r_i$$

for every i . The number x will be constructed by stages and will be given in its decimal representation. At stage i we ensure that x will be distinct from r_i . The number x will be a number between 0 and 1. So the integer part of x is always 0. Here now is the construction.

Stage 0. At this stage we make sure that x will be distinct from r_0 . To do this, we take a decimal representation of r_0 and we let the first fractional digit x_0 of x be the smallest number distinct from the first fractional digit of r_0 . This clearly ensures that the x we want to construct is distinct from r_0 .

Stage i . Assume that we have constructed the first $i - 1$ digits of x . Take r_i . At this stage we make sure that x will be distinct from r_i . To do this, we take a decimal representation of r_i and we let the i th fractional digit x_i of x be the smallest number distinct from the i th fractional digit of r_i . This clearly ensures that the x we want to construct is distinct from r_i .

Now consider the number x , defined as

$$x = 0.x_0x_1x_2x_3 \dots$$

Clearly, x is a real number. At stage i , the construction ensures that $x \neq r_i$ for every i . Therefore, x is distinct from every r_i , $i \in \omega$. We also assumed that the sequence

$$r_0, r_1, r_2, \dots$$

contains all real numbers. Hence x must occur in the sequence. Say $x = r_i$ for some i . We have a contradiction since stage i ensures that x is distinct from r_i . Therefore, the set of all reals cannot be a countable set. The theorem is proved.

Exercise 1.10.7 *Let A be a countable infinite set. Show that the set of all subsets of A is uncountable. (Hint: use the diagonalization method idea).*

1.11 Algorithms

The notion of an algorithm is fundamental to modern mathematics and computer science. In this section we give an informal explanation of algorithms. This explanation will suffice to use this notion later in this book, and to show that some problems have decision algorithms. Let us first consider several examples of known algorithms.

Example 1.11.1 *There exists an algorithm that given a natural number $n \in \omega$ decides if the number is even or odd.*

An example of such an algorithm is the following. Take a number n . Then n is of the form $i_1 \dots i_k$, where $i_1, \dots, i_k \in \{0, 1, 2, \dots, 9\}$. Read the digits of n from left to right. If the last digit of n is either 0 or 2 or 4 or 6 or 8 then n is even. Otherwise n is odd.

Example 1.11.2 *There exists an algorithm that given two integers produces the sum of those integers.*

This is a well-known algorithm that we learn in elementary school. There is no need to explain the algorithm.

Example 1.11.3 *There exists an algorithm that given a number tells us if the number is prime or not.*

An example of such an algorithm is the following. Take a number n . For every $i = 2, \dots, n - 1$, check by turn if n is a multiple of i . If no such i exists, then n is a prime number. Otherwise, n is not a prime number.

Example 1.11.4 *There exists an algorithm that, for any directed graph $\mathcal{G}$ whose vertex set is $\{1, \dots, n\}$ and whose edge set E is a subset of $\{1, \dots, n\} \times \{1, \dots, n\}$, and for some positive integer k , decides if $\mathcal{G}$ has a path of length k .*

Given number k and the graph $\mathcal{G}$, we note that the number of sequences of length k (of nodes of $\mathcal{G}$) is $t = k^n$. An example of a desired algorithm is now the following. List all sequences

$$s_1, s_2, \dots, s_t$$

of length k . For every $i = 1, \dots, t$, check by turn if s_i is a path. If no such s_i exists, then $\mathcal{G}$ does not have a path of length k . Otherwise, $\mathcal{G}$ has a path of length k .

We now informally explain algorithms. In our explanation we use double brackets [[like this]] which refer the reader to the examples above in order to make our explanation clear. Note that our explanation will not give us a formalization of the notion of algorithm. The issue of formalizing the notion of algorithm will not be discussed in this book. However, we will be using algorithms to decide certain problems about some objects (e.g., finite automata, Rabin automata, Büchi automata) which will be introduced later. Thus, an algorithm is a procedure that has the following properties:

- There is a set of inputs to which the algorithm is applied. [[Thus, in Example 1.11.1 and Example 1.11.3 the algorithms are applied to positive integers. The algorithm in Example 1.11.2 is applied to pairs of integers. The algorithm in Example 1.11.4 is applied to pairs of the type (G, k) , where G is a finite graph and k is a positive integer.]]
- The algorithm solves a problem about inputs by giving certain outputs. [[Thus, in Example 1.11.1 the algorithm solves the problem whether or not a given number is even. In Example 1.11.3 the algorithm solves the problem whether or not a given number is prime. In Example 1.11.3 the algorithm solves the problem whether or not a given graph $\mathcal{G}$ has a path of length k . The outputs of these algorithms are “yes” or “no” answers. In Example 1.11.2 the algorithm solves the problem of addition of integers. The outputs of this algorithm are integers.]]

- The algorithm has a finite set of instructions. [[In each of the examples above, algorithms are (or can be) specified by a finite set of instructions.]]
- There is a computer (or someone) who can follow and carry out the instructions. [[In the examples above we could follow and carry out instructions for any given input.]]
- Given an input, the algorithm applied to the input produces a computation which goes by stages. [[Thus, for example, in Example 1.11.1 the stages of a computation on an input n can be described as follows. Given input $n = i_1 \dots i_k$, at stage 1 the computer reads the first digit of the input. At stage 2 the computer reads the second digit of the input. At stage k , the computer reads the last digit of the input. At the final stage the computer makes a decision.]]
- Each stage of the computation does not have elements of randomness.
- There exists a possibility to memorize, calculate, and make logical decisions.

Later in this book we will study certain problems about some objects, e.g., finite state machines, that will be introduced in the next chapters. In many cases we will construct **algorithms that will decide** the problems introduced. In other words, we will provide algorithms that, applied to any object (of our interest), tell us whether or not the object possesses the specifications of the problem. The problems that can be decided by algorithms are called **decidable problems**. For example, the problem whether a positive integer n is prime or not is decidable. Similarly, the problem whether or not a given graph has a path of length k is decidable.

Exercise 1.11.1 *Find algorithms that decide the following problems:*

1. *The problem whether a triple (a, b, c) of integers satisfies the property $a + b = c$,*
2. *The problem whether a graph has a cycle of length k (a cycle of length k in a graph is a path $a_1, a_2, \dots, a_k$ of distinct elements such that the pair (a_k, a_1) is an edge).*
3. *The problem whether a graph is connected.*

For each of these algorithms tell what the inputs, outputs, and instructions are.

Now we are finally ready to begin our journey for studying the theory of computations on finite state machines and applications.